SEng6241:- Mobile Application Development

Chapter -02

Basics of XML, Views & Layouts

Prepared By Seifu D.

Email :- seifiu.detso@wku.edu.et

2026 class

XML, views, and layouts are the ABCs of mobile app design – understanding them deeply empowers developers to create seamless user experiences that resonate with audiences

- ☑ XML Basics
- ☑ Views(UI Controls)
- ☑ Android layouts

XML Basics

What is XML?

- It's stands for eXtensible Markup Language(ML) & MLs are language that uses "markup" in order to encode metadata in a text format. Its not the one and only ML such as SGML, HTML, XHTML .... MLs but XML is different in that: *Unlike other MLs which have predefined tags, XML let you use your own tag.*
  - √ *To make this point clear, take HTML (I hope you all know HTML), HTML allows you to create a web page using predefined tags like <body><form><p><h1> etc*
  - √ *But XML allows you to create a document you like say StudentProfile that has your own tags like; studentname, age, ...*

XML is preferable over other MLs because it:

- ✓ *is self descriptive*
- ✓ *XML is both human (all human languages it support UNICODE) & machine readable*
- ✓ *Can be easily transported with no need for compatibility b/n source & destination*
- ✓ *is platform (OS, CPU architecture, PL, DBMS .....) independent*

Why we study XML here?

XML Basics

Because:

- XML aims to solve some of the long-standing challenges in getting computer systems to interact with each other on a programmatic level.
- Distributed computing has long faced challenges in the way that programming functionality encapsulated within "objects" is exchanged.
- None out of technology has resolved this as much XML had.

+advantages of XML:

- ✓ *It is self descriptive data is the data that describes both its content and structure.*
- ✓ *It is designed to store and transport data.*
- ✓ *Can be easily transported with no need for compatibility b/n source & destination*
- ✓ *it was created to provide an easy to use and store self describing data.*
- ✓ *It is not a replacement for HTML.*
- ✓ *It is designed to carry data, not to display data.*
- ✓ *It tags are not predefined. You must define your own tags.*
- ✓ *It is platform independent and language independent(both).*

XML Document

XML Basics

- HTML is a ML to create a structured document called HTML doc (Web Page).
- Similarly a document produced using XML is called XML-Document.
- Like HTML document has .html extension, XML document has .xml
- unlike HTML documents, which can only be viewed using HTML browsers, XML documents or its part can be processed, displayed, combined to form another, used as input for applications etc.

Example:

```
< proposal>
 <scope>
  < head> work pan </ head>
    < schedule This project begins
        < startdate> April 15 2023</startdate>  and en
        < enddate> june 20 2023 / enddate>
    </schedule>
    < fees> Costs of this proposal are expected to be
         <range>  $160 - $300 < /range>
     </ fees>
  </ scope>
</ proposal>
```

XML Basics

XML document structure

The major portions of an XML document include the following:

- *The XML declaration*
- *The Document Type Declaration*
- *The Element data*
- *The Attribute data*
- *The Character data or XML content*

- *XML documents are allowed to include all or part of the above components,*

1. Start by defining the XML declaration:

<?xml version="1.0 encoding=""?>

What is XML declaration?

1. *It is a preprocessor directive telling your XML file processor (web browser or other application) that this document is XML document.*
2. *Its like of #include lines in C++ programs*

XML Basics

XML document structure

Step 2: declaring the document type definition (DTD)

- *If you have XML document model against which the current document you are creating to be "well formed" or "validated" you will declare the DTD/Schema file in here.*

Step 3: create all Elements/ Tags of the document

- *XML element is the name given for the specific portion of XML document.*
- *A single XML document contains several elements/tags.*

The syntax for creating XML element is :

- ✓ **<element_name> any text </element_name>**
- ✓ **<element_name> and </element_name> are called start and end tag**
- ✓ **element_name is identifier and it is bind by identifier naming rule**

XML Basics

XML document structure

Step 4: create the necessary attributes and sub-elements for each element of the document

- *XML attribute is name-value that contains descriptive info about XML element.*
- *A single XML element may contain zero or many xml sub-elements.*
- *Attribute is defined inside element start tag as: <element attribute="value">*
- *Sub-element of an element is child element of the element it is enclosed in.*

The syntax for creating XML sub-element is :

- ***<element_name><sub-element_name>** value **</sub-element_name>**…..*
  </element_name>
- ***<sub-element_name>** and **</sub-element_name>** are called start and end tag*
- ***sub-element_name** & **attribute_name** are identifier and are abide by identifier naming rule*

XML Basics

XML document structure

Step 5: adding comments and XML ordering

XML comment

its like programming comment is unprocessed illustrative detail.

The XML comment syntax

- To make something a comment write it in between <! and ->
- <! Comment ->

XML Ordering

- Is concerned with the order in which XML document elements should appear.
- it defines the tree like structure of the document being defined.
- An example in next slide shows different XML documents varying by Ordering

XML Basics

XML document structure

Example: creating sub-element and attribute

sampleDic is the xml document type;

it has word element with the ff details

Element is <word>

Head is an Attribute for word element

<Definition> is sub-element for the <word> element

```
<sampleDic>
        <word head = "ethic" >
                <definition>
                rules of behavior based on ideas about what is morally good and bad.
                </definition>
                <definition>
                a set of moral principles, especially ones relating to or affirming a
                specified group, field, or form of conduct.
                </definition>
        </word>
</sampleDic>
```

XML Basics

XML document structure

- *The two documents are different!*
- *For attributes order doesn't matter!*


```
<sampleDic>
        <word>
                <definition>
                rules of behavior based on ideas about
                </definition>
                <source>
                Merriam-Webster, online
                </source>
                <usage>
                Ethics is his chosen field of study.
                </usage>
        </word>
</sampleDic>
```

```
<sampleDic>
        <word>
                <source>
                Merriam-Webster, online
                </source>
                <definition>
                rules of behavior based on ideas about
                </definition>
                <usage>
                Ethics is his chosen field of study.
                </usage>
        </word>
</sampleDic>
```

XML Basics

Rules of XML Structure

1. All XML Elements Must Have a Closing Tag
2. XML Tags Are Case Sensitive
3. All XML Elements Must Have Proper Nesting
4. All XML Documents Must Contain a Single Root Element
5. Attribute Values Must Be Quoted
6. Attributes May Only Appear Once in the Same Start Tag
7. Attribute Values Cannot Contain References to External Entities

XML document structure example

XML Basics

Example01

```
<?xml version="1.0" ?>
<bookstore>
 <book category="COOKING">
    <title lang="en">Everyday Italian</ title >
    <author >Giada De Laurentiis</ author >
    <year>2005</year>
    <price>30.00 </price>
 </book>
 < book category="CHILDREN">
    < title lang="en">Harry Potter</ title >
    < author >J K. Rowling</ author >
    < year >2005</ year >
    < price >29.99</ price >
 </ book >
 <book category="WEB">
    < title lang="en">Learning XML</ title >
    < author >Erik T. Ray</ author >
    < year >2003</ year >
    < price >39.95</price >
 </book>
</bookstore>
```

Example02

```
<?xml version="1.0" ?>
< employee>
    < firstname>J K. Rowling</ firstname>
    < lastname>2005</ lastname>
    < email>29.99</ email>
 </ employee>
```

XML Basics

XML Validation

- *XML file can be validated by 2 ways:*
  1. *against DTD*
  2. *against XSD*

- ✓ *DTD (Document Type Definition) and*
- ✓ *XSD (XML Schema Definition) are used to define XML structure.*

XML Basics

Valid and well-formed XML document with DTD

- Its commonly known as DTD, is a way to describe XML language precisely.
- DTDs check vocabulary and validity of the structure of XML doc against grammatical rules of appropriate XML language.
- It can be either specified inside the doc, or it can be kept in a separate doc

Syntax

Basic syntax of a DTD is as follows −

```
<!DOCTYPE element DTD identifier
   [
      <!ELEMENT root-element(sub-elements)>
      <!ELEMEN sub-element (#PCDATA) >
      declaration1
      declaration2 .....
   ]>
```

There are two types of DTD

1. Internal DTD
2. External DTD

Only difference

```
<!DOCTYPE root-element
SYSTEM "file-name"
```

In the above syntax,

- The **DTD** starts with <!DOCTYPE delimiter.
- An **element** tells the parser to parse the document from the specified root element.
- **DTD identifier** is an identifier for the DTD, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called **External Subset.**
- **The square brackets [ ]** enclose an optional list of entity declarations called **Internal Subset**

XML Basics Cont...

Example internal DTD document

```
<?xml version="1.0" encoding="UTF-8" ?
  standalone="yes">

<!DOCTYPE address
  [
        <!ELEMENT address(name,company,phone)>
        <!ELEMENT name (#PCDATA)>
        <!ELEMENT company (#PCDATA)>
        <!ELEMENT phone (#PCDATA)>
  ]>

      <address>
        <name>Wanos</name>
        <company>Iwin Acadamy</company>
        <phone>(+2519) 123-4567</phone>
      </address>
```

Example external DTD document

address.dtd

```
<!ELEMENT address (name,company,phone)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT company (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
```

address.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<!DOCTYPE address SYSTEM "address.dtd">
     <address>
          <name>wanos zet</name>
          <company>Iwin Acadamy</company>
          <phone>(+2519) 123-4567</phone>
     </address>
```

*The **address internal DTD** is said to be well-formed as :*

- *It defines the type of document by **element** type.*
- *It includes a root element named as **address**.*
- *Each of the child elements among name, company and phone is enclosed in its self explanatory tag.*
- *Order of the tags is maintained.*

XML Basics

Cont...

Try this one!

Create a well-documented and valid XML document with a Document Type Definition (DTD), incorporating <message> elements with 'to,' 'from,' 'heading,' and 'body' attributes. Ensure proper structure and validation. Explain your DTD choices and how they maintain document integrity and compliance with XML standards.

Message internal DTD xml document with DTD

```
<?xml version="1.0"?>
<!DOCTYPE message[
<!ELEMENT message (to, from,
heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<message>
   <to>Tove</to>
         <from>Jani</from>
         <heading>Reminder</heading>
         <body>Don't forget me this
weekend</body>
</message>
```

XML Basics

2. Checking Validation of XML Schema definition

- It is used to describe and validate the structure and the content of XML data.
- XML schema defines the elements, attributes and data types.
- Schema element supports Namespaces. It is similar to a database schema that describes the data in a database.
- An XML document is called "well-formed" if it contains the correct syntax.
- A well-formed and valid XML document is one which have been validated against Schema.

The basic idea behind XML Schemas is that they describe the legitimate format that an XML document can take.

XML Schema Data types

- There are two types of data types in XML schema.
  - simpleType
  - complexType

simpleType

- The simpleType allows you to have text-based elements. It contains less attributes, child elements, and cannot be left empty.

complexType

- The complexType allows you to hold multiple attributes and elements. It can contain additional sub elements and can be left empty.

XML Basics

Cont...

Syntax
You need to declare a schema in your XML document as follows

```
<?xml version = "1.0" encoding = "UTF-8"?>
  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name = "contact">
     <xs:complexType>
       <xs:sequence>
          <xs:element name="name" type="xs:string" />
          <xs:element name="company" type= "xs:string" />
          <xs:element name = "phone" type = "xs:int" />
        </xs:sequence>
     </xs:complexType>
    </xs:element>
  </xs:schema>
```

```
<xs:attribute name = "x" type = "y"/>
```

Description of XML Schema

- *<xs:element name="contact">: It defines the element name contact.*
- *<xs:complexType>: It defines that the element contact' is complex type.*
- *<xs:sequence>: It defines that the complex type is a sequence of elements.*
- *<xs:element name="name" type="xs:string"/>: It defines that the element 'name' is of string/text type.*
- *<xs:element name="company" type="xs:string"/>: It defines that the element 'company' is of string/text type.*
- *<xs:element name="phone" type="xs:int"/>: It defines that the element 'phone' is of integer/numeric type.*

XML Basics Cont...

XML Validation using XML Schema

```
<?xml version="1.0" encoding = "UTF-8"?>
<employee xmlns="http://www.iwinacadamy.com"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.iwinacdamy.com employee.xsd">
  <firstname> wanos </firstname>
  <lastname> zet</lastname>
  <email> wanoszet@gmail.com</email>
</employee>
```

employee.xml


```
<?xml version = "1.0" encoding = "UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.javatpoint.com"
  xmlns="http://www.javatpoint.com">
    <xs:element name="employee">
     <xs:complexType>
     <xs:sequence>
        <xs:element name="firstname" type="xs:string"/>
        <xs:element name="lastname" type="xs:string"/>
        <xs:element name="email" type="xs:string"/>
      </xs:sequence>
     </xs:complexType>
    </xs:element>
</xs:schema>
```

employee.xsd

XML Basics

DTD Vs XSD

No.	DTD	XSD
1)	DTD stands for **Document Type Definition**.	XSD stands for XML Schema Definition.
2)	DTDs are derived from **SGML** syntax.	XSDs are written in XML.
3)	DTD **doesn't support datatypes**.	XSD **supports datatypes** for elements and attributes.
4)	DTD **doesn't support namespace**.	XSD **supports namespace**.
5)	DTD **doesn't define order** for child elements.	XSD **defines order** for child elements.
6)	DTD is **not extensible**.or **limited extensibility**	XSD is **extensible**.
7)	DTD is **not simple to learn.**.	XSD is **simple to learn** because you don't need to learn new language..
8)	DTD provides **less control** on XML structure.	XSD provides **more control** on XML structure.

Summery on XML document

In generally, XML allows creating document model in two types of modeling

1. ***Freeform XML Modeling:*** *in this type there are minimal rules in the model & you have the right to use any name for tags and tags can appear in any order. Document created as such is called* ***well-formed*** *(****legal XML syntax and structure according to the XML specification****)XML Document.*
2. ***Strict-XML Modeling:*** *If all documents to be created [known as document instances] has to follow all the rules, tag precedence and tag naming when validated against the one specified in the Document Model, the created instance is called* ***Valid XML*** *(****one that is well-formed and conforms to a structure outlined in a DTD or schema****) Document.*

Clear so far?

"

End of XML Basics

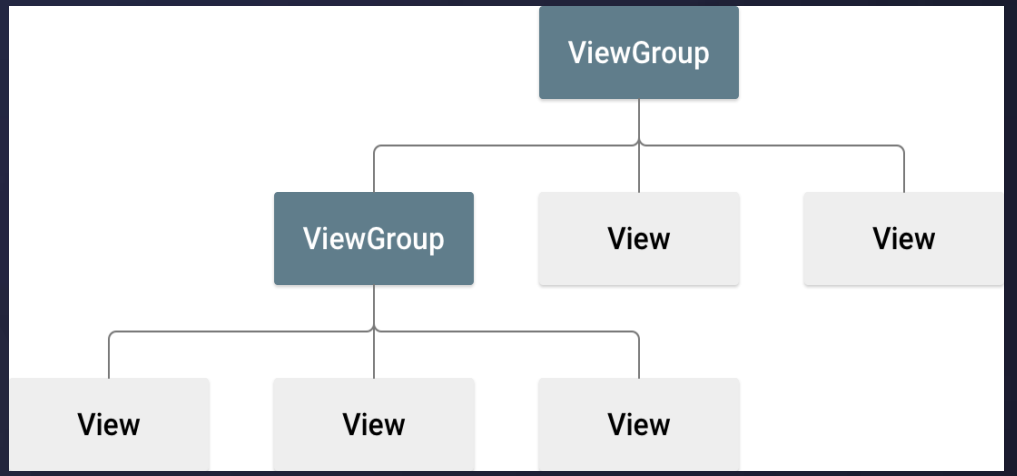
CONTENTS

- XML Basics
- Views(UI Controls)
- Android layouts

UI Controls

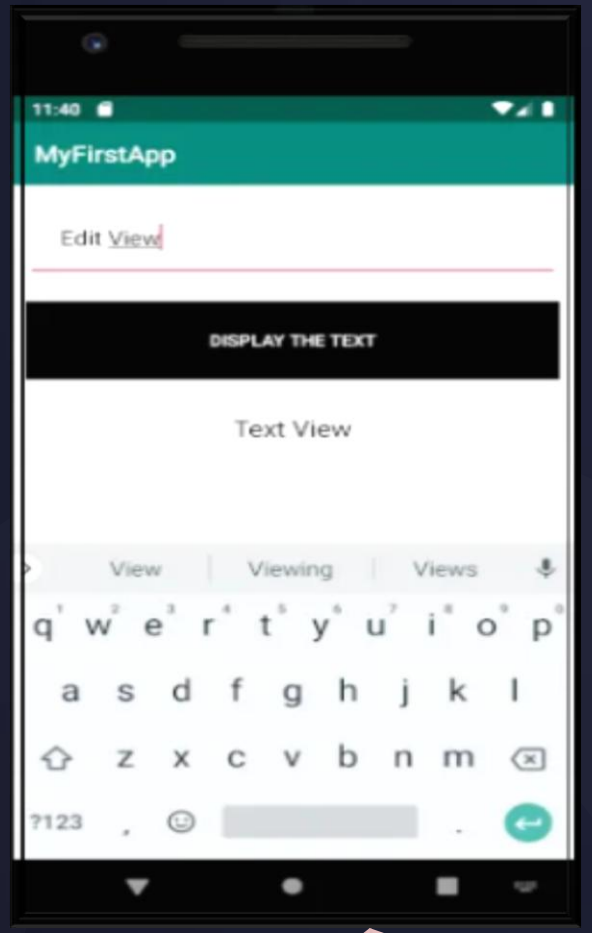
- *An Android application UI is everything that the user can see and interact with.*
- *A **View** is an object that draws something on the screen that the user can interact with and*
- ***ViewGroup** is an object that holds other View (and ViewGroup) objects in order to define the layout of the UI.*
- *All user interface elements in an Android app are built using View and ViewGroup objects.*
- *A ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects.*

*The **ViewGroup** objects are usually called **layouts** and can be one of many types that provide a different layout structure, such as **LinearLayout** , **Relativelayout** or **ConstraintLayout.***



- *A **layout** defines the structure for a user interface in your app, such as in an activity.*
- *View objects are often called widgets and can be one of many subclasses, such as Button or TextView.*

UI Controls: Example



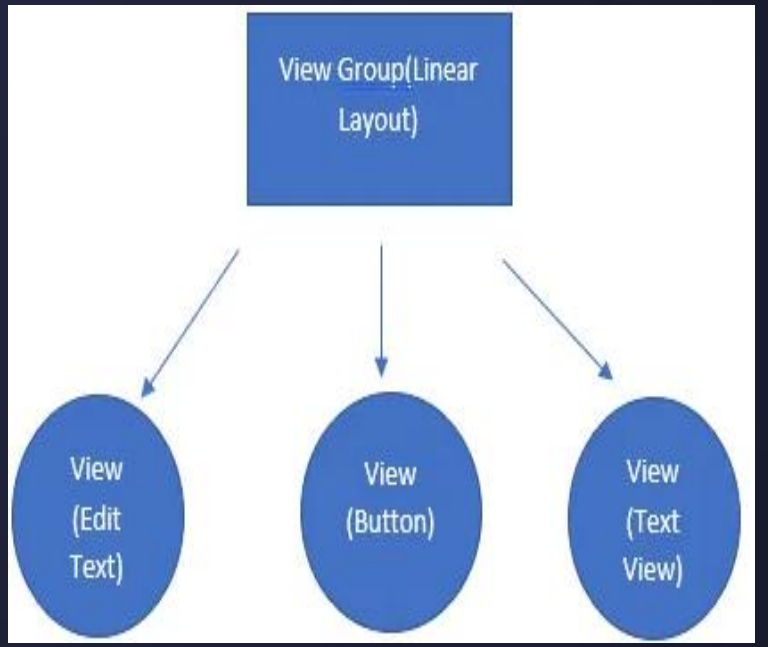
App UI on mobile screen

```
<EditText
    android:id="@+id/writtentText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter any string"
    android:padding="25dp"
    android:layout_margin="10dp"
    />

<Button
    android:id="@+id/btn"
    android:layout_width="match_parent"
    android:layout_height="70dp"
    android:background="#000"
    android:text="Display the Text"
    android:textColor="#fff"
    android:layout_margin="10dp"/>

<TextView
    android:textColor="#000"
    android:layout_margin="20dp"
    android:textSize="20sp"
    android:text="Text"
    android:gravity="center"
    android:id="@+id/displayText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Apps UI XML desgin



Hierarchal structure

Methods to Create a UI

- *There are two ways to create a user interface (UI) in Android: **declaratively** and **programmatically**.*
- *They are quite different but often are used together to get the job done.*

1. Declarative User Interface

- *The declarative approach involves using XML to declare what the UI will look like,*
- *similar to creating a web page using HTML. You write tags and specify elements to appear on your screen.*
- *If you have ever hand coded an HTML page, you did pretty much the same work as creating an Android screen*

- *Advantage:*
  - *XML is fairly human-readable, and even people who are unfamiliar with the Android platform and framework can readily determine the intent of the user interface.*
- *The disadvantage*
  - *it doesn't provide a good way of handling user input. That's where the programmatic approach comes in.*

Methods to Create a UI

2. Programmatic User Interface

- *A programmatic user interface involves writing Java code to develop the UI.*
- *If you have ever done any Java AWT or Java Swing or javafx development, Android is pretty much the same in that respect. It is similar to many UI toolkits in other languages as well.*
- *Everything you can do on declaratively, you can also do programmatically.*
- *But Java also allows you to specify what happens when that button is actually clicked. This is the main advantage of a programmatic approach to the user interface.*

- *Basically, if you want to create a button programmatically,*
  - *declare the button variable*
  - *create an instance of it*
  - *add it to a container*
  - *and set any button properties that may make sense, such as color, text, text size, background, and so on.*

So which approach to use?

✓ *The best practice is to use both you'd use XML to declare what the "button" looks like and Java to specify what it does.*

Programmatic UI layout: Example

Views are building blocks of Activities/UI
Create using Programmatic UI layout

main.xml

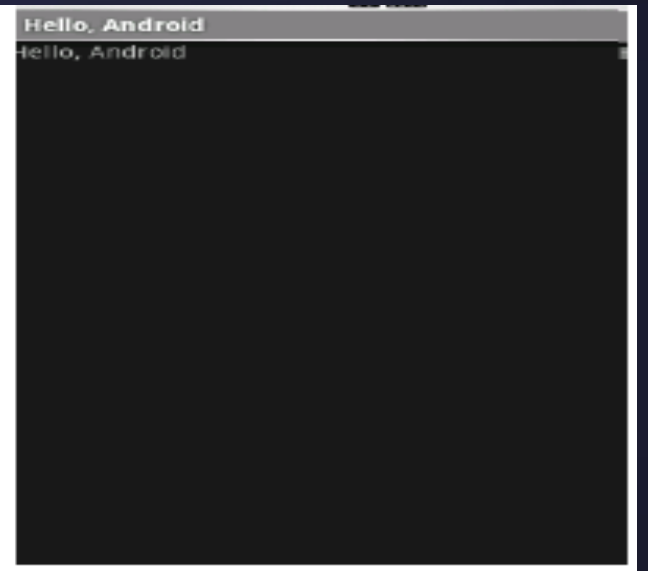
```
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:text="@string/hello"
</TextView>
```

XML-based UI layout file

MyActivity.java

```
public class MyActivity extends Activity
{
public void onCreate(Bundle savedInstanceState)
  {
     super.onCreate(savedInstanceState);
     setContentView(R.layout.main);
  }
}
```

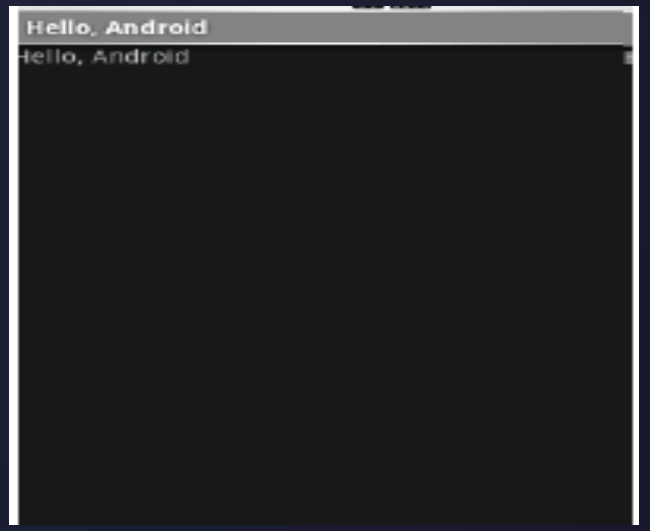
15



MyActivity.java

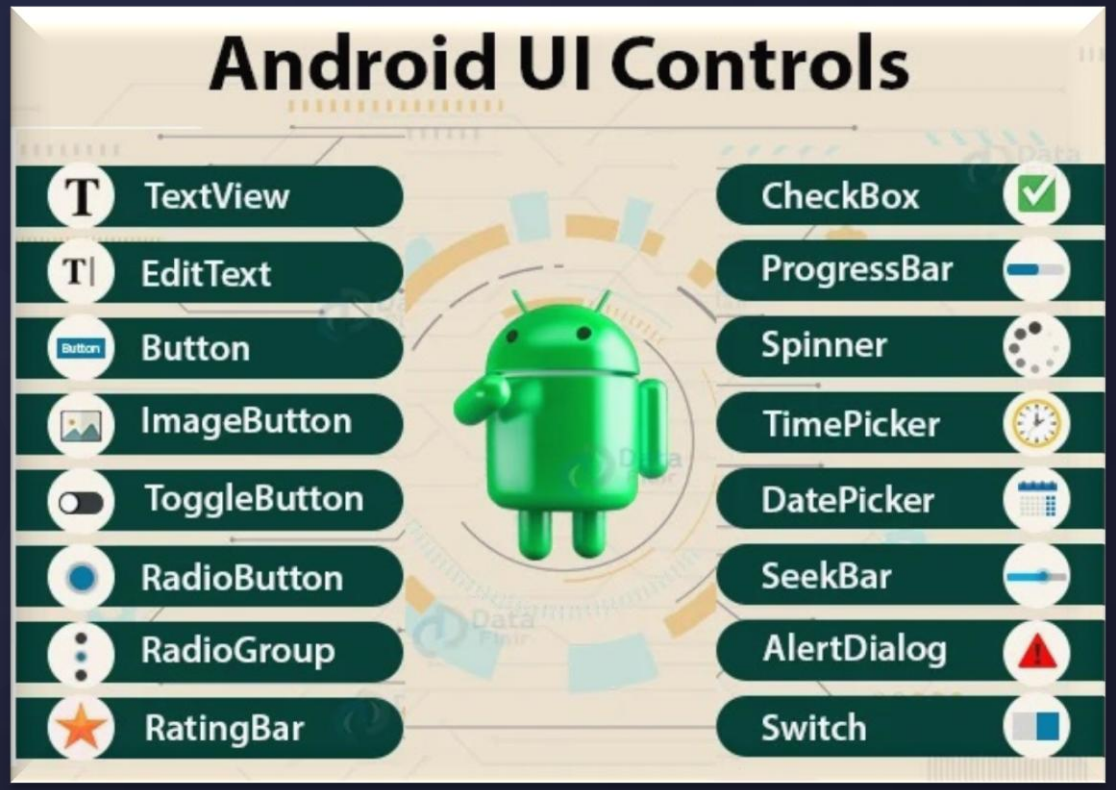
```
public class MyActivity extends Activity
{
public void onCreate(Bundle savedInstanceState)
  {
     super.onCreate(savedInstanceState);
    // setContentView(R.layout.main);
     TextView tv = new TextView(this);
     tv.setText("Hello, Android");
     setContentView(tv);

  }
}
```



Common UI Controls

Control Type	Description	Related Classes
Button	A push-button that can be pressed, or clicked, by the user to perform an action.	`Button`
Text field	An editable text field. You can use the `AutoCompleteTextView` widget to create a text entry widget that provides auto-complete suggestions	`EditText`, `AutoCompleteTextView`
Checkbox	An on/off switch that can be toggled by the user. You should use checkboxes when presenting users with a group of selectable options that are not mutually exclusive.	`CheckBox`
Radio button	Similar to checkboxes, except that only one option can be selected in the group.	`RadioGroup` `RadioButton`
Toggle button	An on/off button with a light indicator.	`ToggleButton`
Spinner	A drop-down list that allows users to select one value from a set.	`Spinner`
Pickers	A dialog for users to select a single value for a set by using up/down buttons or via a swipe gesture. Use a `DatePicker`code> widget to enter the values for the date (month, day, year) or a `TimePicker` widget to enter the values for a time (hour, minute, AM/PM), which will be formatted automatically for the user's locale.	`DatePicker`, `TimePicker`



- *Input controls are the **interactive components** in your app's user interface. Android provides a wide variety of controls you can use in your UI, such as **buttons, text fields, seek bars, checkboxes, zoom buttons, toggle buttons**, and many more.*

Event Listener (Input Events)

- *An interface, in the **View** class, that contains a single callback method*
- *Called by the Android framework when the View is triggered*
- *Multiple Event Listeners (Interfaces):*
  - *View.OnClickListener - onClick()*
  - *View.OnLongClickListener - onLongClick()*
  - *View.OnFocusChangeListener - onFocusChange()*
  - *View.OnKeyListener - onKey()*
  - *View.OnTouchListener - onTouch()*
  - *View.OnCreateContextMenuListener – onCreateContextMenu()*

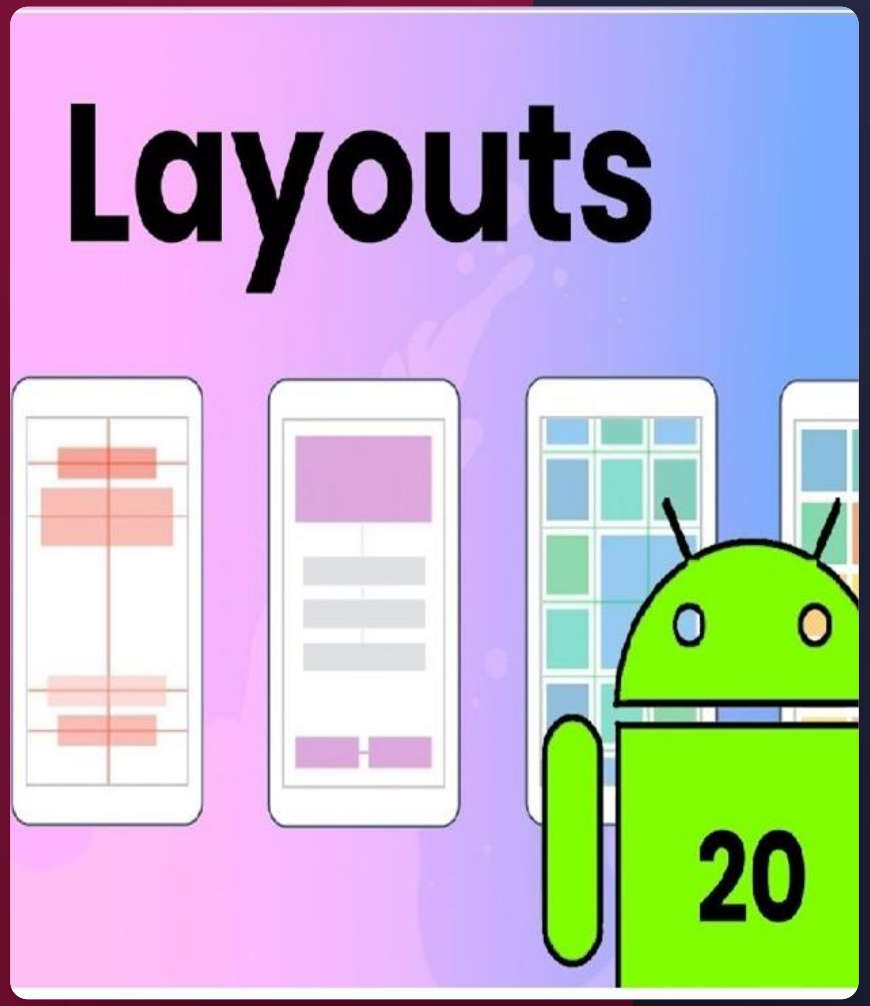
```
public class ExampleActivity extends Activity implements OnClickListener {
    protected void onCreate(Bundle savedValues) {
        ...
        Button button = (Button)findViewById(R.id.button_1);
        button.setOnClickListener(this);
    }

    // Implement the OnClickListener callback
    public void onClick(View v) {
      switch (v.getId()) {
      case R.id.button_1:
            // do action when the button is clicked
            break;
       // handle more buttons in the activity
       }
    }
    ...
}
```

Example: onClick() for Button view

Clear so far?

End of UI Controls



CONTENTS

- XML Basics
- Views(UI Controls)
- Android layouts

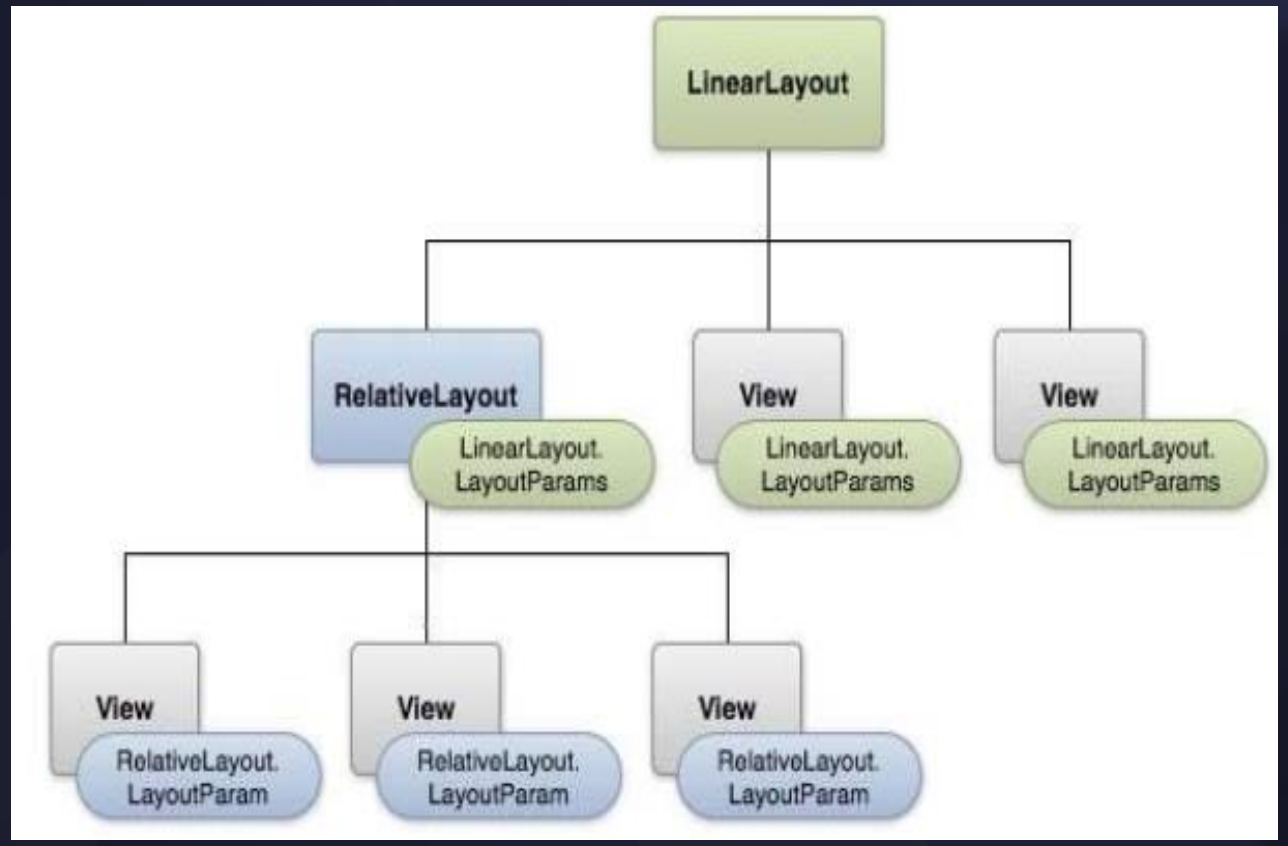
Android layouts

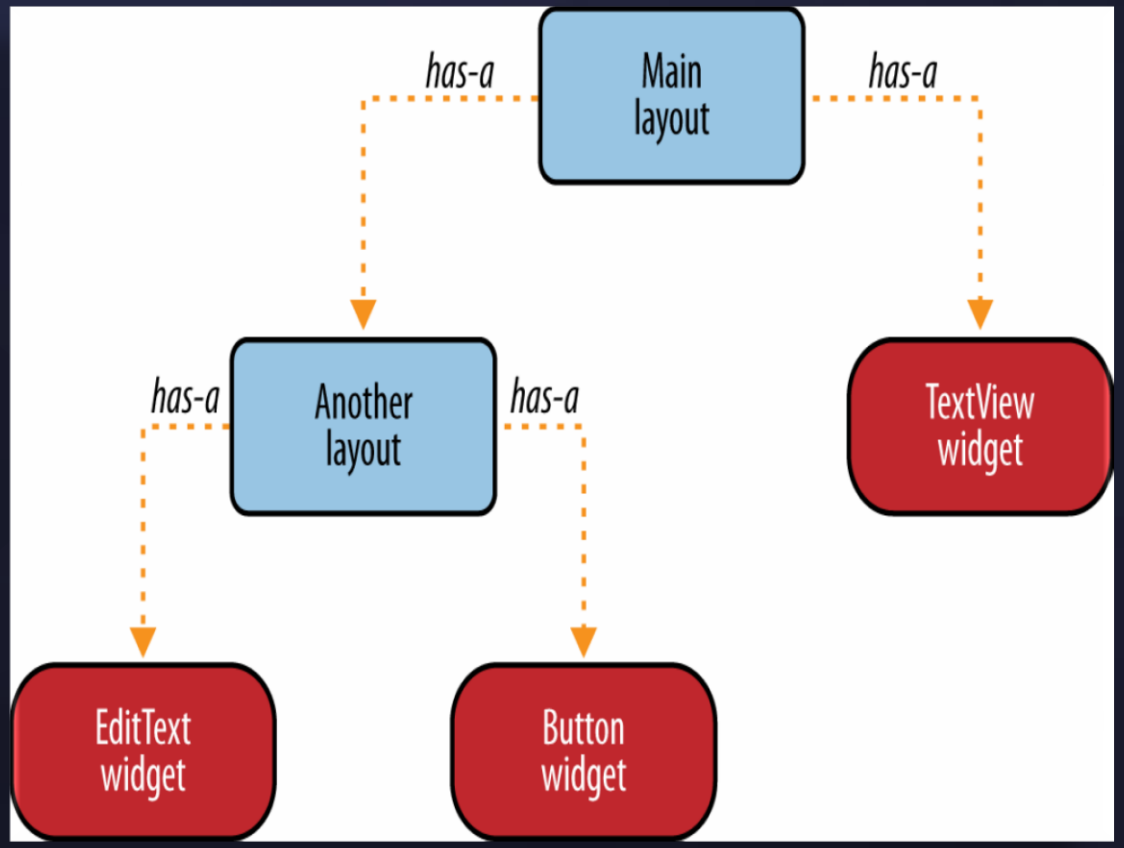
- *Android organizes its UI elements into views and layouts.*
- *Everything you see, such as a button, label, or text box, is a view.*
- *Layouts organize views, such as grouping together a button and label or a group of these elements.*
- *If you have prior experience with Java Swing, layouts are similar to Java containers and views are similar to Java components.*
- *Views in Android are sometimes referred to as widgets.*

- *A layout can contain other children. Those children can furthermore be layouts themselves, allowing for a complex user interface structure.*
- *A layout is responsible for allocating space for each child.*
- *A layout defines the visual structure for a user interface, such as the UI for an activity or app widget.*
  - ✓ *Can be specified in*
  - ✓ *res/layout/activity_main.xml*
  - ✓ *Each Activity can have it's own layout*

```
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);     }
```

Android layouts





Hierarchal structure

Common Layouts

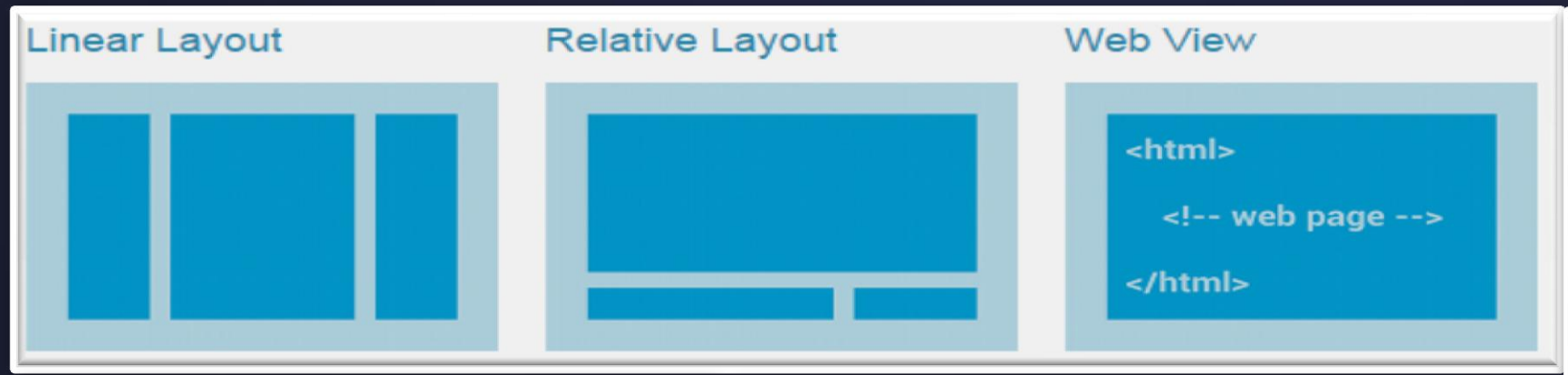
- ***Linear Layout**: is a view group that aligns all children in a single direction, vertically or horizontally.*
- ***Relative Layout**: is a view group that displays child views in relative positions. Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).*
- ***Table :Layout** is a view that groups views into rows and columns.*
- ***Absolute Layout** : enables you to specify the exact location of its children.*
- ***List View**: is a view group that displays a list of scrollable items.*
- ***Grid View**: is a ViewGroup that displays items in a two-dimensional, scrollable grid.*
- ***Frame Layout**: is a placeholder on screen that you can use to display a single view.*

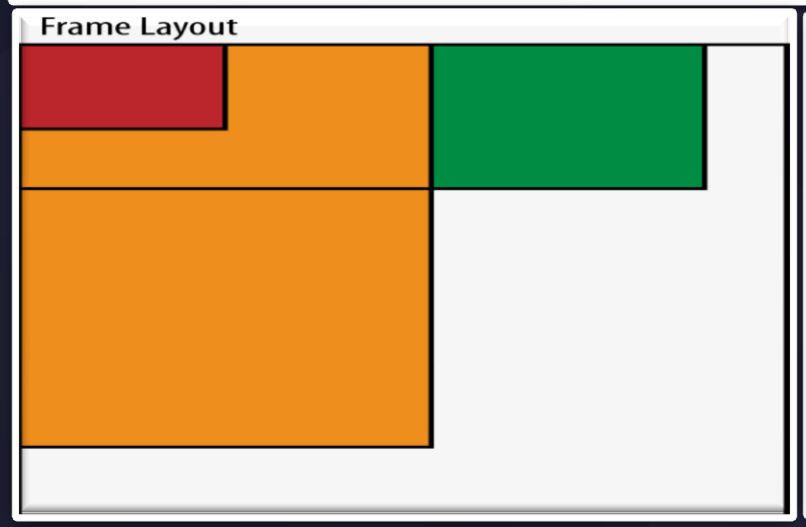
Here are some common layouts used in Android applications:

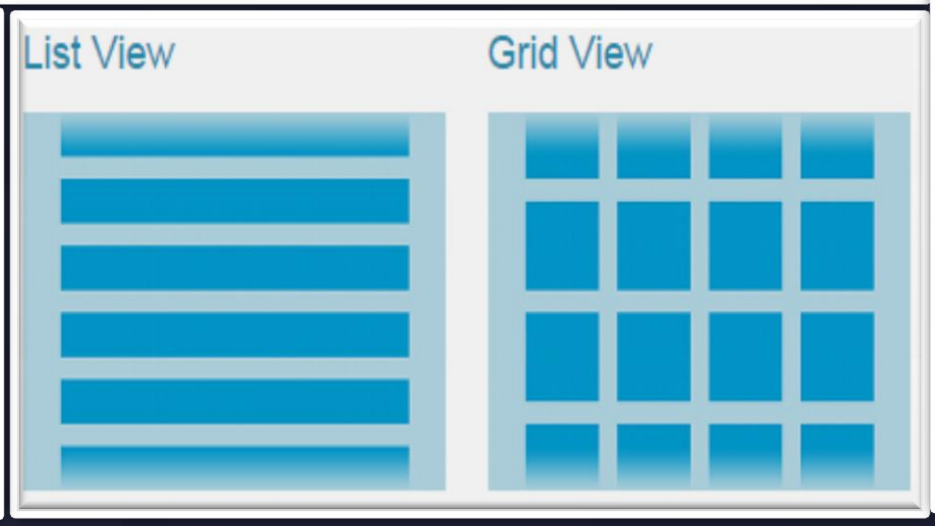
1. *Linear Layout*
2. *Relative Layout*
3. *Constraint Layout*
4. *Frame Layout*
5. *Table Layout*
6. *Grid Layout*
7. *Scroll View*
8. *Drawer Layout*
9. *Coordinator Layout*
10. *Toolbar Layout*
11. *Card View*
12. *List View / Recycler View*
13. *Tab Layout*
14. *Navigation Drawer Layout*
15. *Fragment Layout*
16. *Dialog Layout*

Common Layouts

Web View: *Displays web pages.*

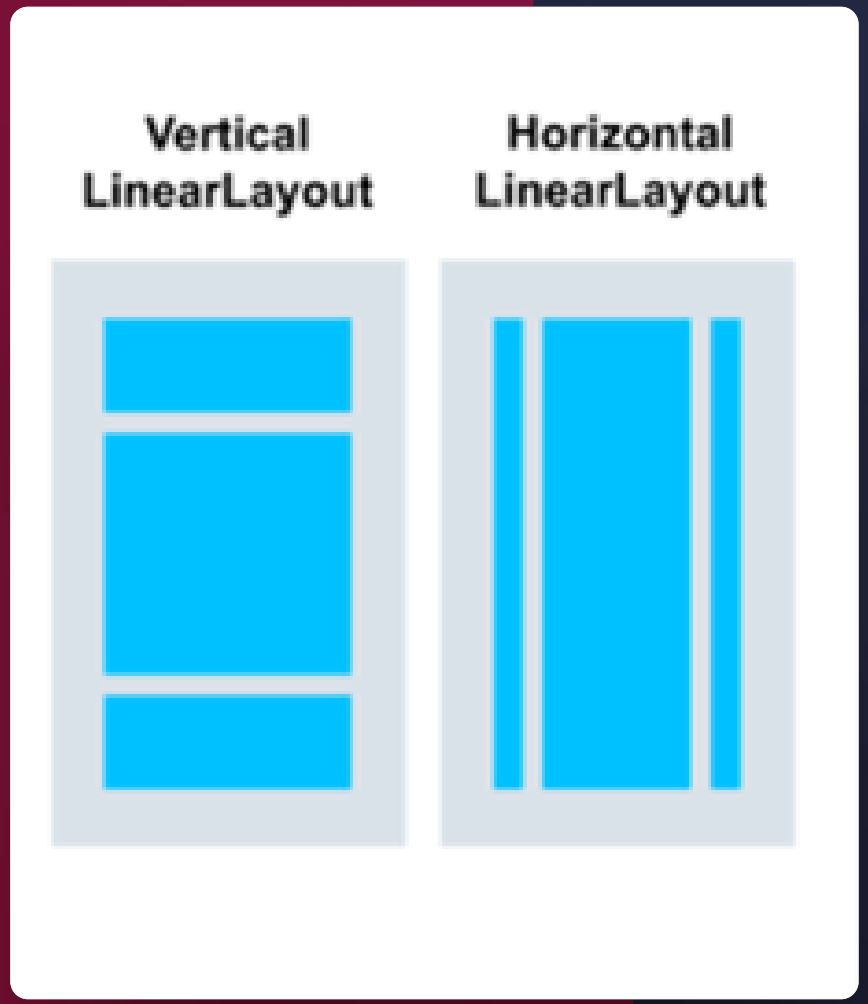






Common Attributes

Attribute	Description	Notes
layout_width	Specifies the width of the View or ViewGroup	
layout_height	Specifies the height of the View or ViewGroup	
layout_marginTop	Specifies extra space on the top side of the View or ViewGroup	
layout_marginBottom	Specifies extra space on the bottom side of the View or ViewGroup	
layout_marginLeft	Specifies extra space on the left side of the View or ViewGroup	
layout_marginRight	Specifies extra space on the right side of the View or ViewGroup	
layout_gravity	Specifies how child Views are positioned	Only in LinearLayout or TableLayout
layout_weight	Specifies the ratio of Views	Only in LinearLayout or TableLayout

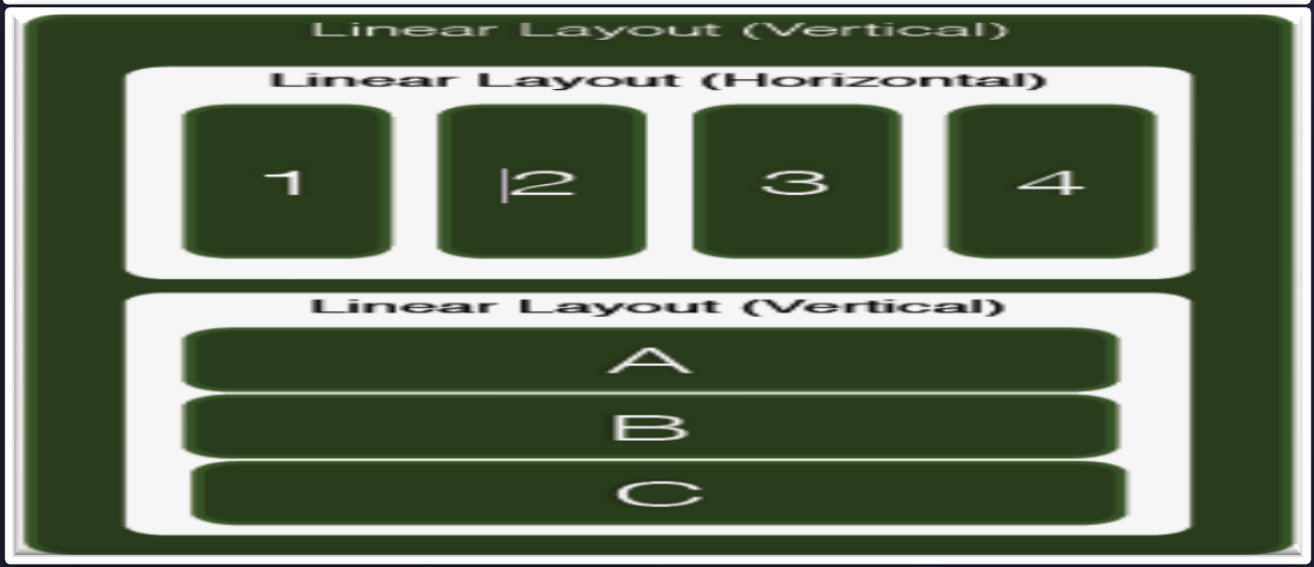


LinearLayout

LinearLayout

- *LinearLayout is one of the simplest and most common layouts*
- *A Layout that arranges its children in a single column or a single row.*
- *Available space is divided among layout children*
- *The direction of the row "android:orientation="vertical"".*
- *You can also specify gravity, which specifies the alignment of all the child elements using XML*

  android:gravity="center_horizontal"
- *The default orientation is horizontal.*

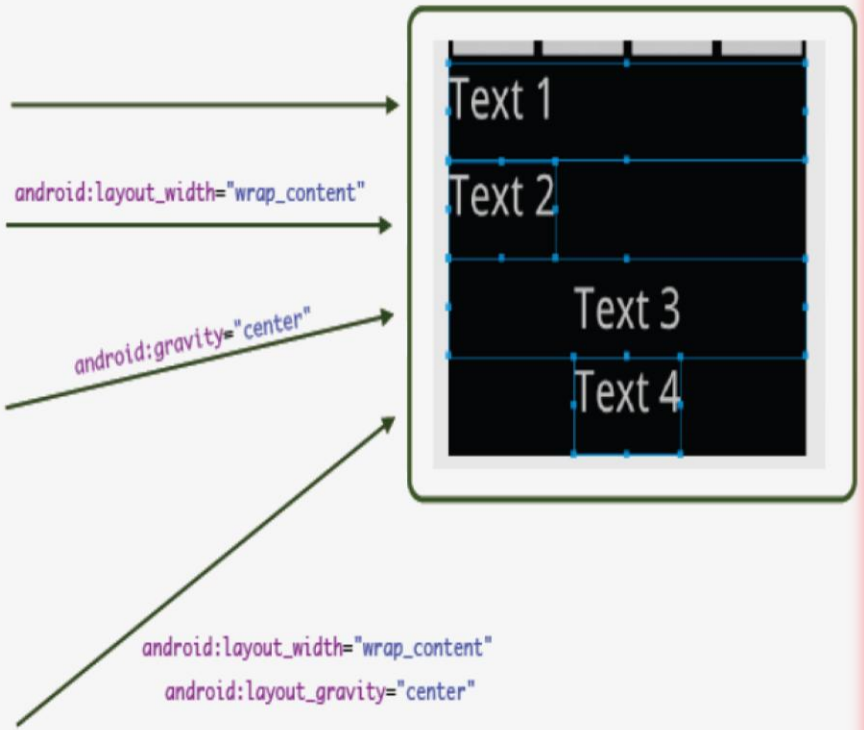


Linear Layout Attributes

Attribute	Description
android:id	This is the ID which uniquely identifies the layout.
android:baselineAligned	This must be a boolean value, either "true" or "false" and prevents the layout from aligning its children's baselines.
android:divider	This is drawable to use as a vertical divider between buttons. You use a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".
android:gravity	This specifies how an object should position its content, on both the X and Y axes. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.
android:orientation	This specifies the direction of arrangmet and you will use "horizontal" for a row, "vertical" for a column. The default is horizontal.

LinearLayout

```
<LinearLayout
  android:orientation="vertical"
  android:layout_width="fill_parent"
  android:layout_height="fill_parent"
  android:layout_weight="1">
  <TextView
      android:text="Text 1"
      android:textSize="15pt"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:layout_weight="1"/>
  <TextView
      android:text="Text 2"
      android:textSize="15pt"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_weight="1"/>
  <TextView
      android:text="Text 3"
      android:textSize="15pt"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:gravity="center"
      android:layout_weight="1"/>
  <TextView
      android:text="Text 4"
      android:textSize="15pt"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_gravity="center"
      android:layout_weight="1"/>
</LinearLayout>
```



UI Design challenge#01

☺ ***Contact Information:***

- ***Views:*** *TextViews (for name, phone number, email), ImageView (for contact picture).*
- ***Layout:*** *LinearLayout (vertical).*
- ***Explanation:*** *Create a contact information layout displaying a person's details vertically. The linear arrangement suits well for a single-column format*

LinearLayout

UI Design challenge#02

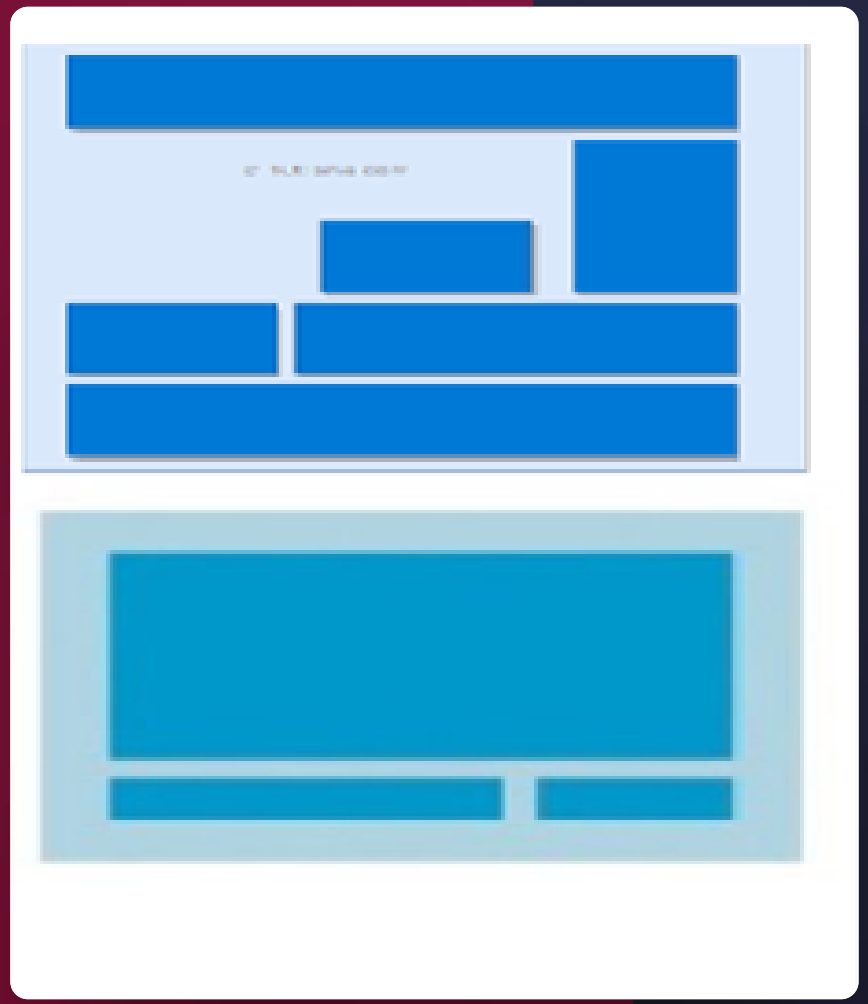
☺ ***Music Playlist:***

- ***Layout***: *LinearLayout (horizontal).*
- ***Views***: *ImageView (album cover), TextView (song title and artist), Button (play).*
- ***Explanation***: *Create a music playlist with horizontally arranged song details and play button using LinearLayout.*

UI Design challenge#03

☺ ***News Headlines:***

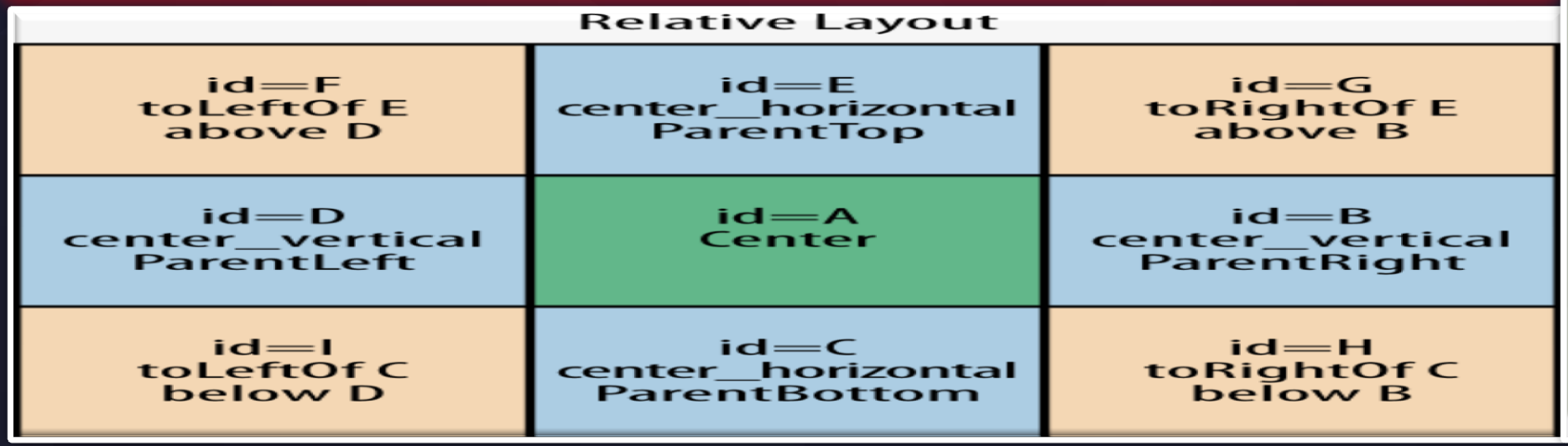
- *Layout: LinearLayout (vertical).*
- *Views: TextView (for news headlines), Button (for read more).*
- *Explanation: Display a list of news headlines with read more buttons in a vertically stacked layout using LinearLayout.*

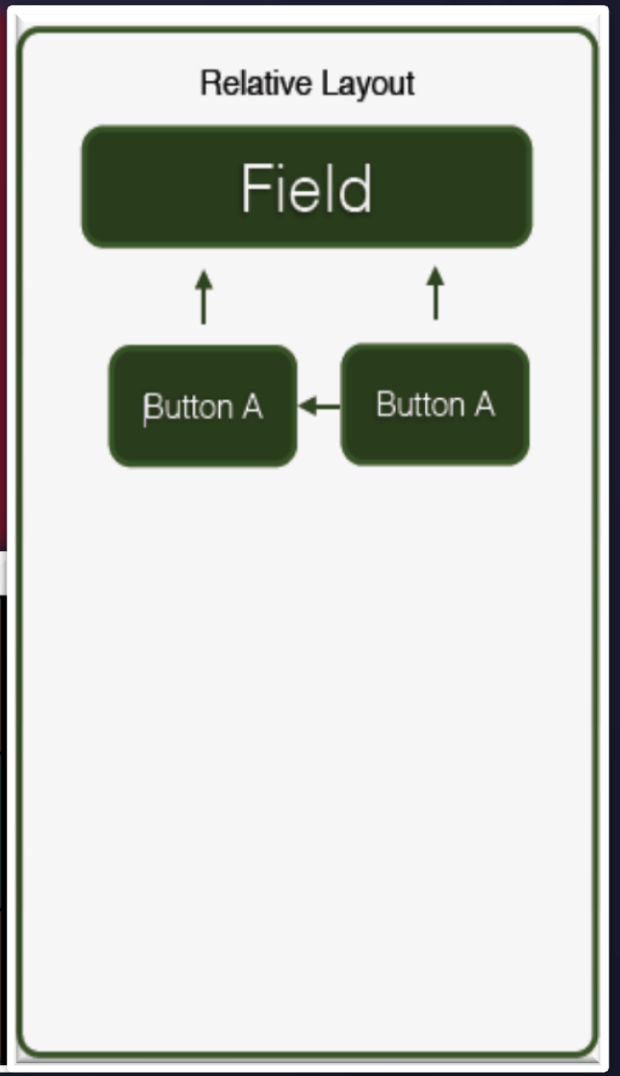


RelativeLayout

RelativeLayout

- *It is a ViewGroup that displays child View elements in relative positions.*
- *It is very powerful because it doesn't require you to nest extra layouts to achieve a certain look.*
- *The position of a View can be specified as relative to sibling elements (such as to the left-of or below a given element) or in positions relative to the RelativeLayout area (such as aligned to the bottom, left of center).*





Relative Layout


```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
    <TextView
        android:id="@+id/label"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Type here:"/>
    <EditText
        android:id="@+id/entry"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:background="@android:drawable/editbox_background"
        android:layout_below="@id/label"/>
    <Button
        android:id="@+id/ok"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/entry"
        android:layout_alignParentRight="true"
        android:layout_marginLeft="10dip"
        android:text="OK" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/ok"
        android:layout_alignTop="@id/ok"
        android:text="Cancel" />
</RelativeLayout>
```

Relative Layout

UI Design challenge#01

☺ ***Login Page:***

- *Layout: RelativeLayout(RL).*
- *Views: EditText (username and password), Button (login), TextView (forgot password).*
- *Explanation: Create a login page with username, password fields, login button, and a forgot password link positioned relative to each other in RL.*

UI Design challenge#02

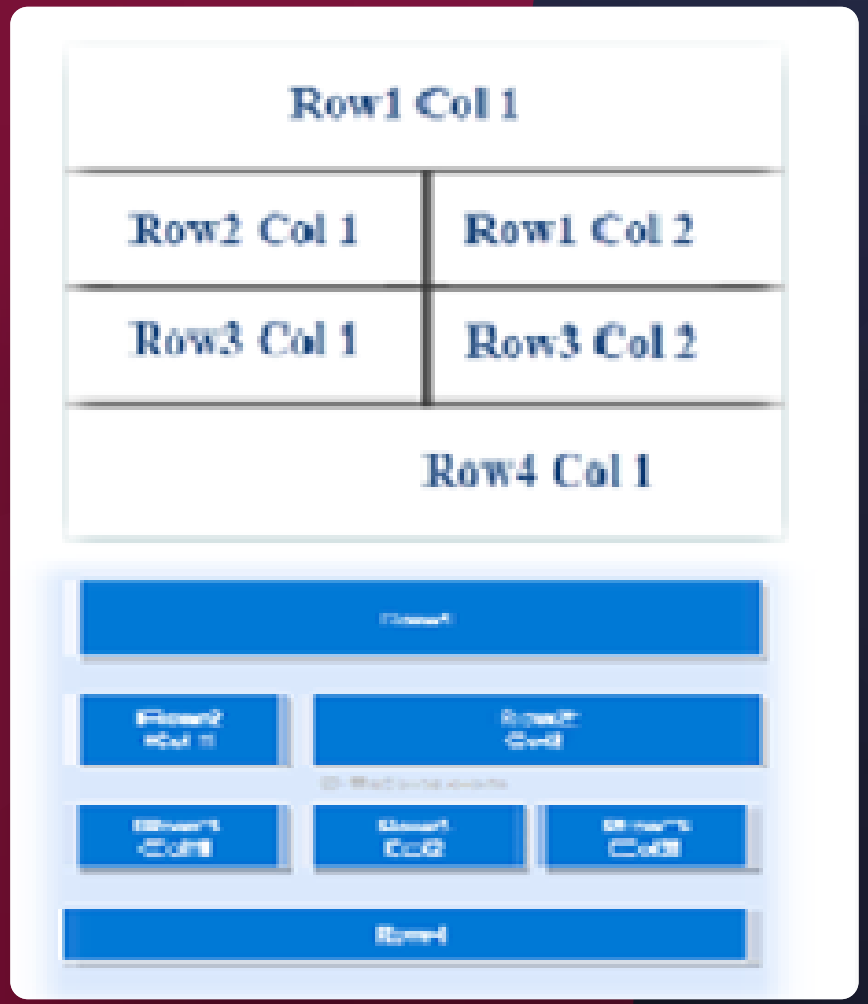
☺ ***Weather Forecast:***

- *Layout: RelativeLayout.*
- *Views: TextView (for temperature), ImageView (weather icon), TextView (for city name).*
- *Explanation: Create a weather app with temperature, weather icon, and city name arranged relative to each other within a RL.*

UI Design challenge#03

☺ ***Chat Conversation:***

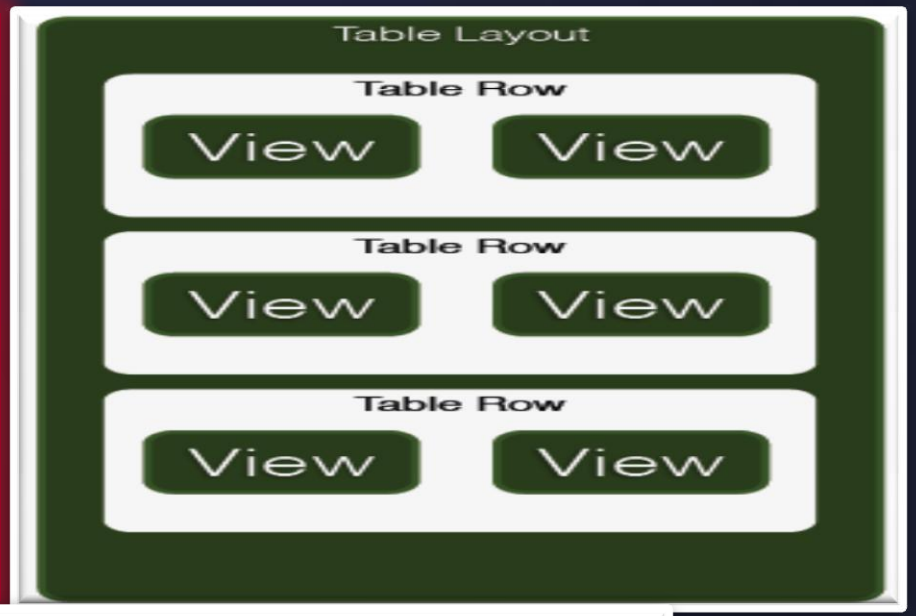
- *Layout: RelativeLayout.*
- *Views: TextView (chat messages), EditText (message input), Button (send).*
- *Explanation: Implement a chat application with chat messages, message input field, and send button positioned relative to each other in a RelativeLayout*

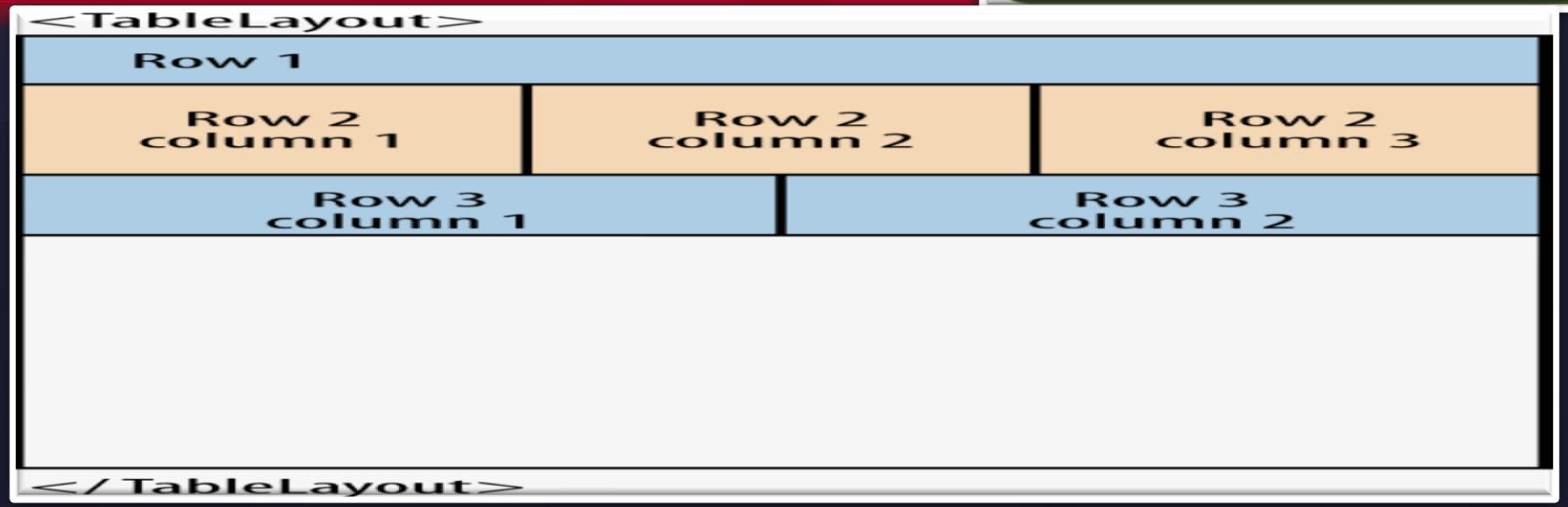


TableLayout

TableLayout

- *TableLayout is a ViewGroup that displays child View elements in rows and columns.*
- *Has a structure similar to an HTML table.*
- *The TableLayout element is like the HTML <table> element; TableRow is like a <tr> element.*
- *In each cell you can use any kind of View element arranged like columns with horizontal linear layout.*

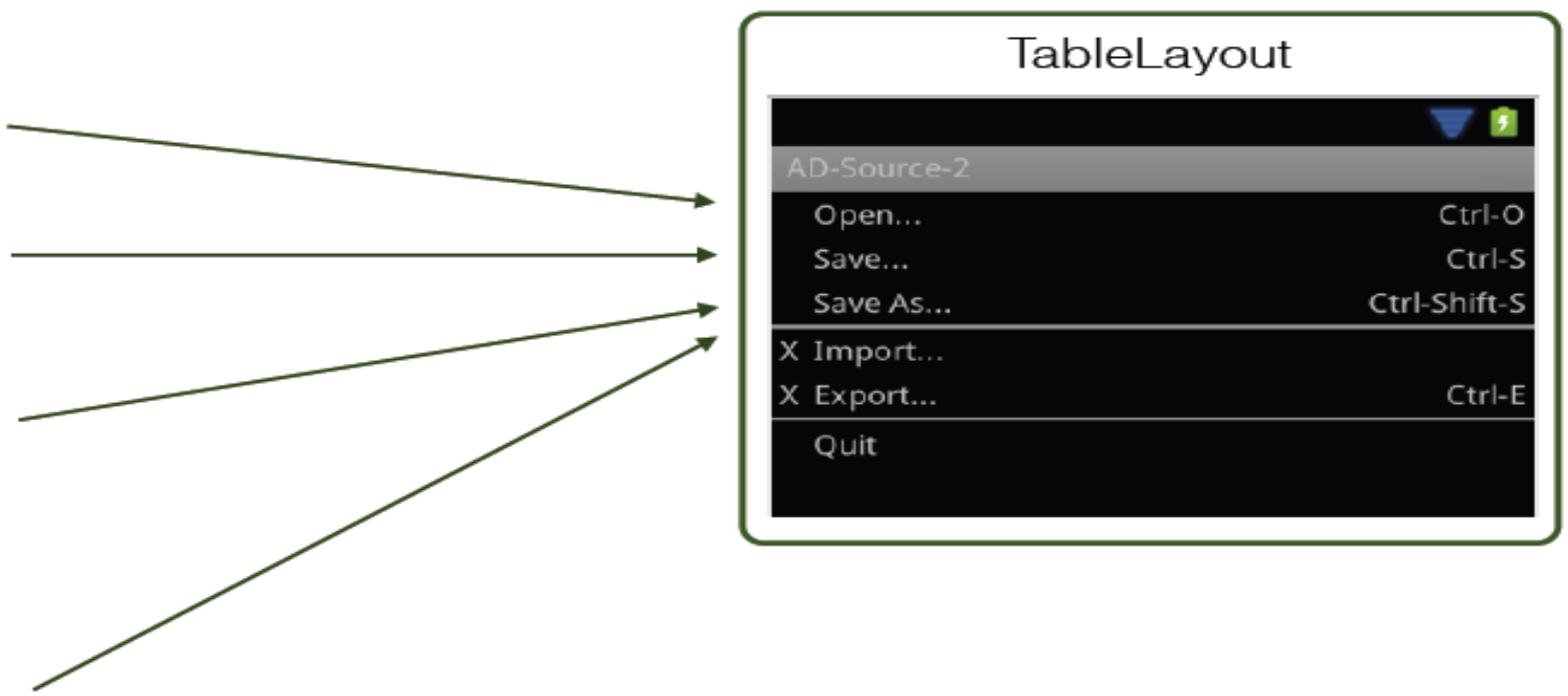




TableLayout

```
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
android:layout_height="fill_parent"
    android:stretchColumns="1">
    <TableRow>
        <TextView
            android:layout_column="1"
            android:text="Open..."
            android:padding="3dip" />
        <TextView
            android:text="Ctrl-O"
            android:gravity="right"/>
    </TableRow>
    <TableRow>
        <TextView
            android:layout_column="1"
            android:text="Save..." />
        <TextView
            android:text="Ctrl-S"
            android:gravity="right"/>
    </TableRow>
    <TableRow>
        <TextView
            android:layout_column="1"
            android:text="Save As..."/>
        <TextView
            android:text="Ctrl-Shift-S"
            android:gravity="right"/>
    </TableRow>
    <View
        android:layout_height="2dip"
        android:background="#FF909090" />
.....

</TableLayout>
```



Defining Styles

```
...
<TableRow android:id="@+id/TableRow01"
                android:layout_weight="0.2"
                android:layout_width="fill_parent"
                android:layout_height="wrap_content">
      <Button android:id="@+id/Button07"
                 style="@style/mybutton" android:text="7" />
      <Button android:id="@+id/Button08"
                 style="@style/mybutton" android:text="8" />
      <Button android:id="@+id/Button09"
                  style="@style/mybutton" android:text="9" />
      <Button android:id="@+id/ButtonDivide"
                 style="@style/mybutton" android:text="/" />
</TableRow>
...
```

Defining Styles

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<style name="mybutton" parent="@android:style/TextAppearance">
        <item name="android:textSize">30sp</item>
        <item name="android:textColor">#000000</item>
        <item name="android:layout_width">fill_parent</item>
        <item name="android:layout_height">fill_parent</item>
        <item name="android:layout_weight">1</item>
    </style>
</resources>
```

- *Way of building and formatting layout to your app*
- *Style has formatting attributes to several elements*
- *Theme has formatting attributes to all activities*
  - *Create the theme in xml (styles.xml)*
  - *Apply the theme in manifest*

```
<application android:theme="@style/CustomTheme">
```

```
<activity android:theme="@android:style/Theme.Dialog">
```

TableLayout

UI Design challenge#01

☺ ***Language Translation List:***

- *Layout: TableLayout.*
- *Views: TextViews (for source language, target language, translation).*
- *Explanation: Build a language translation app displaying translations in a table format. Rows represent translation pairs, and columns represent the source language, target language, and translation text.*

UI Design challenge#02

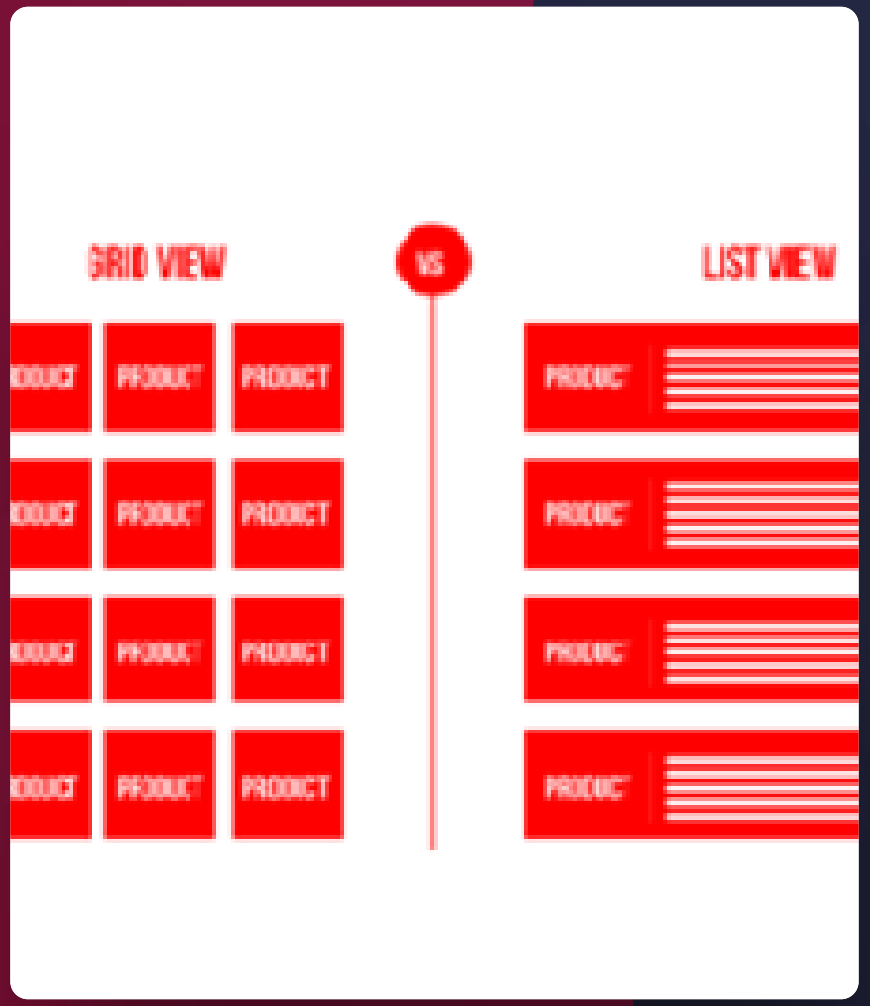
☺ ***Weather Flight Timetable:***

- *Layout: TableLayout.*
- *Views: TextViews (for flight numbers, destinations, departure times).*
- *Explanation: Develop a flight timetable where each row represents a flight, and columns display flight numbers, destinations, and departure times.*

UI Design challenge#03

☺ ***Sports Scores Table:***

- *Layout: TableLayout.*
- *Views: TextViews (for team names, scores, match dates).*
- *Explanation: Create a sports scores table displaying recent match results. Each row represents a match, with columns for team names, scores, and the date of the match.*



List View

ListView and GridView

- *When the content for your layout is dynamic or not pre-determined, you can use a layout that subclasses AdapterView to populate the layout with views at runtime.*
- *A subclass of the AdapterView class uses an Adapter to bind data to its layout.*
- *The Adapter behaves as a middleman between the data source and the AdapterViewlayout—the Adapter retrieves the data (from a source such as an array or a database query) and converts each entry into a view that can be added into the AdapterView layout.*
- *Common layouts backed by an adapter include:*
  - *List View : Displays a scrolling single column list.*
  - *Grid View: Displays a scrolling grid of columns and rows.*

ListView

- *Android **ListView** is a view which groups several items and display them in vertical scrollable list.*
- *The list items are automatically inserted to the list using an **Adapter** that pulls content from a source such as an array or database.*
- *An **adapter** actually bridges between UI components and the data source that fill data into UI Component.*
- ***Adapter** can be used to supply the data to like spinner, list view, grid view etc.*
- *The **ListView** and **GridView** are subclasses of **AdapterView** and they can be populated by binding them to an **Adapter**, which retrieves data from an external source and creates a View that represents each data entry.*
- *The two most common adapters are **ArrayAdapter** and **SimpleCursorAdapter**.*
- *We will see separate examples for both the adapters.*

GridView

- *Android GridView shows items in two-dimensional scrolling grid (rows & columns) and the grid items are not necessarily predetermined b*
- *An adapter actually bridges between UI components and the data source that fill data into UI Component.*
- *Adapter can be used to supply the data to like spinner, list view, grid view etc.*
- *The ListView and GridView are subclasses of AdapterView and they can be populated by binding them to an Adapter, which retrieves data from an external source and creates a View that represents each data entry.*
- *they automatically inserted to the layout using a ListAdapter*

ListView

UI Design challenge#01

☺ ***Exam Results:***

- *Views: ListView.*
- *Data Source: List of student names and their exam scores.*
- *Explanation: Display exam results where each item in the ListView represents a student's name and their corresponding score. Users can scroll through the list to view different students' results*

UI Design challenge#02

☺ ***Messaging App Inbox:***

- *Views: ListView.*
- *Data Source: List of messages (sender, message, timestamp).*
- *Explanation: Design a messaging app inbox where each item in the ListView represents a received message. Users can scroll through messages, tap to view the full conversation, and reply.*

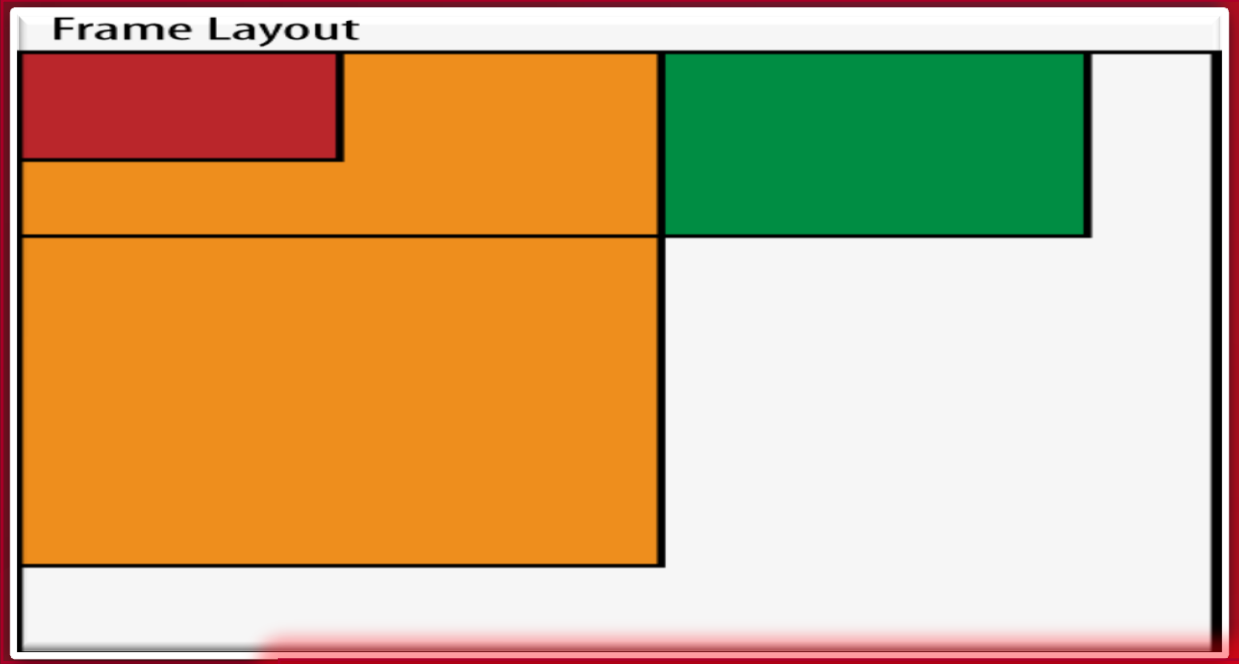
UI Design challenge#03

☺ ***City Guide:***

- *Views: ListView.*
- *Data Source: List of places of interest (name, description, image).*
- *Explanation: Create a city guide app where each item in the ListView represents a place of interest. Users can scroll through places, tap to view details, and navigate to selected places*

Frame Layout

FrameLayout



UI Design challenge#01

☺ *Image Gallery:*

- *Layout: FrameLayout.*
- *Views: ImageView (for displaying images), ImageButton (for navigation to next/previous images), TextView (for image captions).*
- *Explanation: Create an image gallery where the ImageView displays the current image, and navigation buttons and captions are overlaid on top using FrameLayout*

UI Design challenge#02

☺ *Splash Screen :*

- *Layout: FrameLayout.*
- *Views: ImageView (for app logo), ProgressBar (for loading indicator), TextView (optional for app name).*
- *Explanation: Use FrameLayout to overlay a splash screen with the app logo, a loading indicator, and the app name. The loading indicator can be positioned on top of the logo while the app is initializing.*

Scroll View

ScrollView

UI Design challenge#01

Android Registration Form Design Task Description:

Objective: *Design a user-friendly and visually appealing registration form for an Android application. The form should include a variety of user interface elements to collect different types of user input. Your task is to implement these elements within a ScrollView to ensure optimal user experience across various screen sizes.*

Requirements:

User Interface Elements:

- *EditText: For entering the user's name and email address.*
- *CheckBox: To confirm acceptance of terms and conditions.*
- *RadioButton and RadioGroup: For selecting gender (Male/Female).*
- *ToggleButton: To allow users to subscribe/unsubscribe.*
- *RatingBar: To gather feedback by allowing users to rate their experience.*
- *Spinner: For selecting the user's country.*
- *DatePicker: To choose a date.*
- *TimePicker: To select a specific time.*
- *SeekBar: For adjusting progress or any other numerical value.*
- *Switch: To toggle notification preferences.*
- *Registration Progress Bar (ProgressBar):*

Layout Design:

- *Use a ScrollView to accommodate all UI elements, ensuring the form is accessible on smaller screens and include a clear and informative title at the top of the form (e.g., "**Registration Form**") to indicate the purpose of the form to users.*

THANKYOU

Next Topics

- Activities
- Intents
- Services
- Providers
- Receivers.
- Application context

Under chapter 03